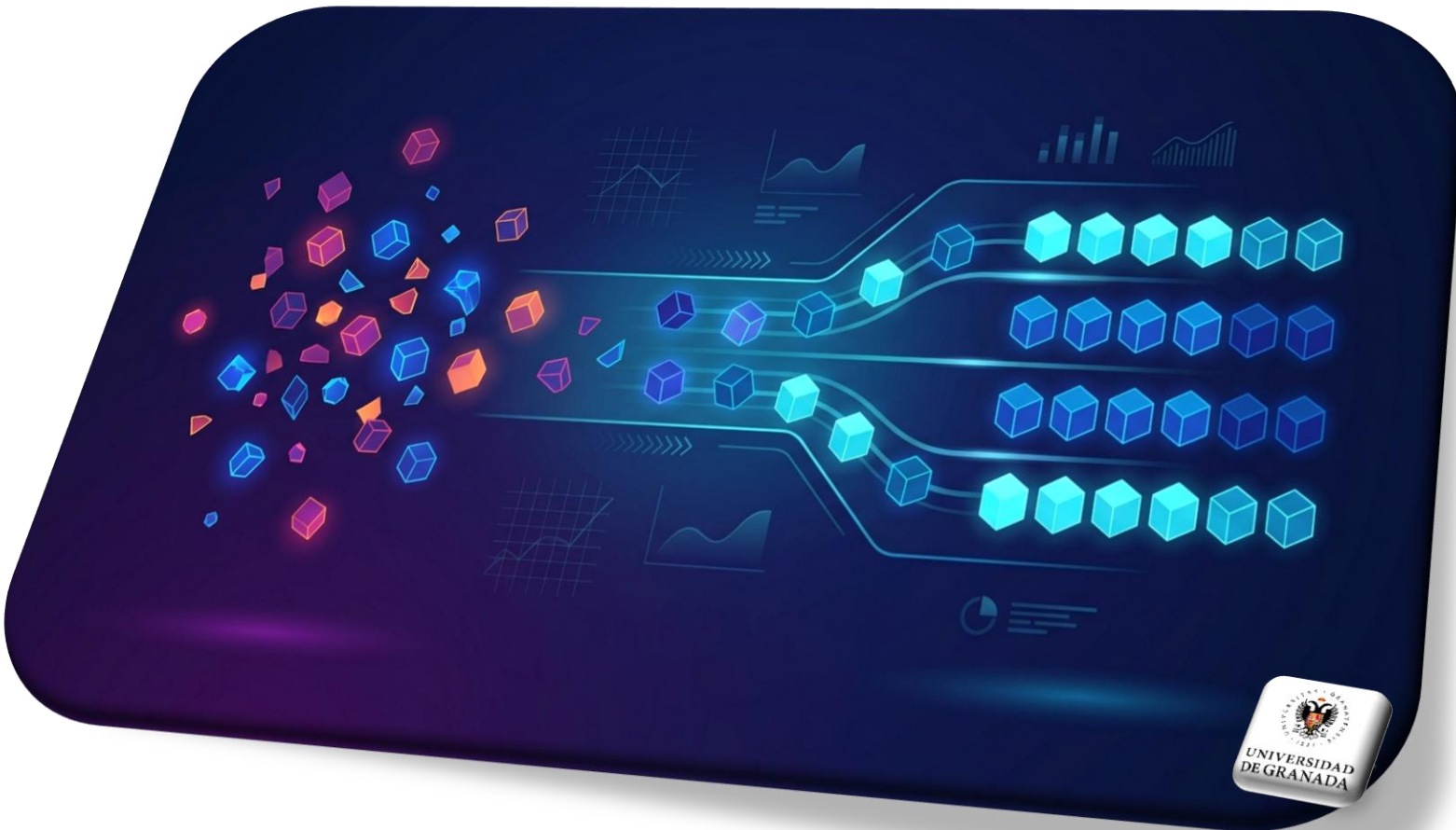


PRÁCTICA 1

ANÁLISIS DE EFICIENCIA DE ALGORITMOS

Algorítmica, Universidad de Granada



Realizado por (C1-3):

- Jaime Espín Rodríguez,
- Marvin Augusto Fonseca Zamora,
- Omar Segura Arévalo,
- Francisco González Bailón,
- Alejandro Moreno Millán.

ÍNDICE

1. Introducción	3
2. Entorno	3
3. Análisis Empírico.....	4
3.1. Cuadrático	5
3.2. Lineal – Logarítmico.....	6
3.3. Cúbico	7
3.4. Exponencial.....	8
3.5. Algoritmos de ordenación.....	9
4. Eficiencia Híbrida	10
4.1. Cuadrático	10
4.2. Lineal – Logarítmico.....	11
4.3. Cúbico	12
4.4. Exponencial.....	13
4.5. Ajuste Incorrecto.....	14
5. Principio de Invarianza	15
6. Conclusiones	16
7. Anexo	18

1. Introducción

El objetivo principal de esta práctica consiste en analizar empíricamente un conjunto de algoritmos de distinta eficiencia para poder validar su complejidad asintótica teórica (Notación O).

Más allá del análisis teórico, usaremos herramientas matemáticas, en este caso *gnuplot*, para calcular la eficiencia híbrida de los algoritmos y poder ver las constantes ocultas (a_0) de cada algoritmo, gracias al ajuste por mínimos cuadrados.

Adicionalmente, pondremos a prueba el principio de invarianza, observando cómo factores externos (optimización, entorno, hardware...) impactan en el tiempo de ejecución real sin alterar el orden de complejidad de los algoritmos.

A partir de los mencionados experimentos, extraeremos las conclusiones que relacionen el comportamiento matemático teórico con las limitaciones físicas del hardware.

2. Entorno

Para garantizar que las medidas para los distintos algoritmos sean en las mismas condiciones, hemos optado por utilizar un ordenador base para ejecutar todos los algoritmos (*ordenador1*), mientras que para poder comparar los resultados y poder comprobar el principio de invarianza hemos usado dos ordenadores más (*ordenador2* y *ordenador3*). A continuación, mencionamos información relevante sobre los respectivos ordenadores debido a su importante relación con las constantes ocultas:

- *Ordenador1*: Sistema operativo Windows, utilizando un procesador Intel i7 con 8 GB de RAM.
- *Ordenador2*: Sistema operativo Windows, utilizando un procesador Intel i5 con 16 GB de RAM.
- *Ordenador3*: Sistema operativo Windows, utilizando un procesador Intel i9 con 16 GB RAM.

Para las imágenes y tablas de esta práctica, se asume que son datos relacionados con el *Ordenador1*, a no ser que se indique lo contrario.

Además del ordenador elegido, debemos de tener en cuenta que todos los resultados que se mostrarán, si no se indica lo contrario, han sido compilados con la versión veinte de C++ (*-std=c++20*), sin ninguna optimización y utilizando como compilador g++.

3. Análisis Empírico

Para realizar las mediciones de los distintos algoritmos hemos decidido usar la librería `<chrono>` debido a que utiliza un reloj de alta precisión del ordenador. Para no tener que repetir código en todos los algoritmos, hemos decidido realizar una clase *myTime.h* que encapsule las operaciones necesarias para obtener el tiempo transcurrido, así el código de los algoritmos nos quedaría así:



```
#include "myTime.h"

int main(){
    MyTime timer;

    timer.start();
    algoritmo();
    timer.end();

    cout << "El tiempo transcurrido es " << *timer << " segundos.\n";
    return 0;
}
```

Tenemos que realizar para cada algoritmo diferentes ejecuciones, variando la cantidad de elementos (N) siguiendo las indicaciones del guion de prácticas. Además, para evitar que las medidas se vean afectadas por factores externos (llamadas al sistema, interrupciones, actualizaciones) se realizan k iteraciones para cada variación de N, tomando como resultado final la media aritmética de los tiempos. Este valor k se ha ajustado dependiendo de la familia de eficiencia teórica del algoritmo.

Nos ha resultado más fácil realizar las mediciones en el propio código C++ del algoritmo, en vez de realizar un script externo al código. Es por ello, que la medición de los algoritmos se podría interpretar como dos bucles anidados de la siguiente forma:

```

ejemplo.cpp

#include "myTime.h"

double algoritmo(const int N){
    MyTime timer;
    timer.start();
    //Operaciones
    timer.end();
    return *timer;
}

int main(){
    const int k = 1; //Varía según el orden de eficiencia
    double sumatorio = 0.0;

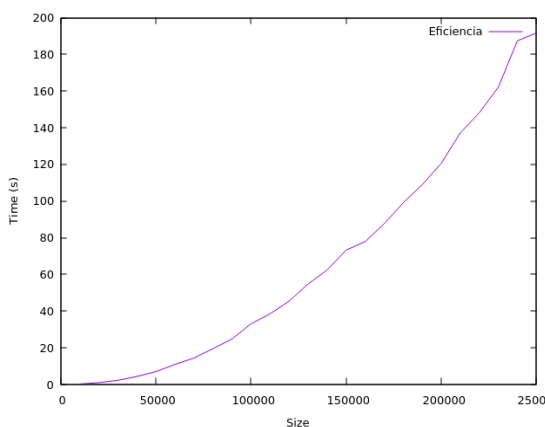
    for(int n = INICIO; n <= FINAL; n += SALTO){
        for(int i = 0; i < k; i++){
            sumatorio += algoritmo(n);
        }
        cout << "El tiempo es " << sumatorio / double(k);
        sumatorio = 0.0;
    }

    return 0;
}

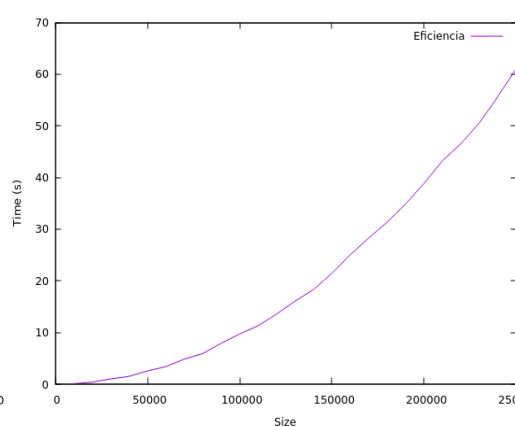
```

A continuación, mostraremos las gráficas independientes de cada algoritmo, así como una gráfica general para cada familia de eficiencia. Las tablas con los datos pertinentes se encuentran en la sección [Anexo](#).

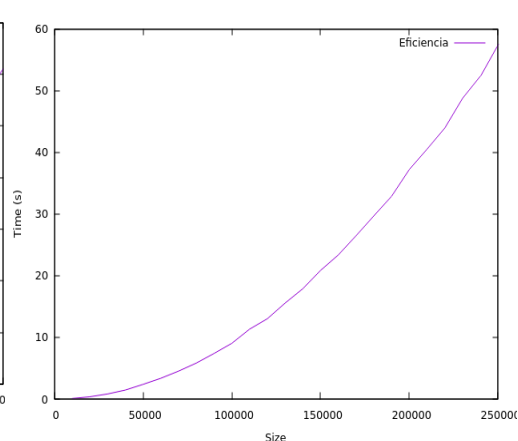
3.1.Cuadrático:



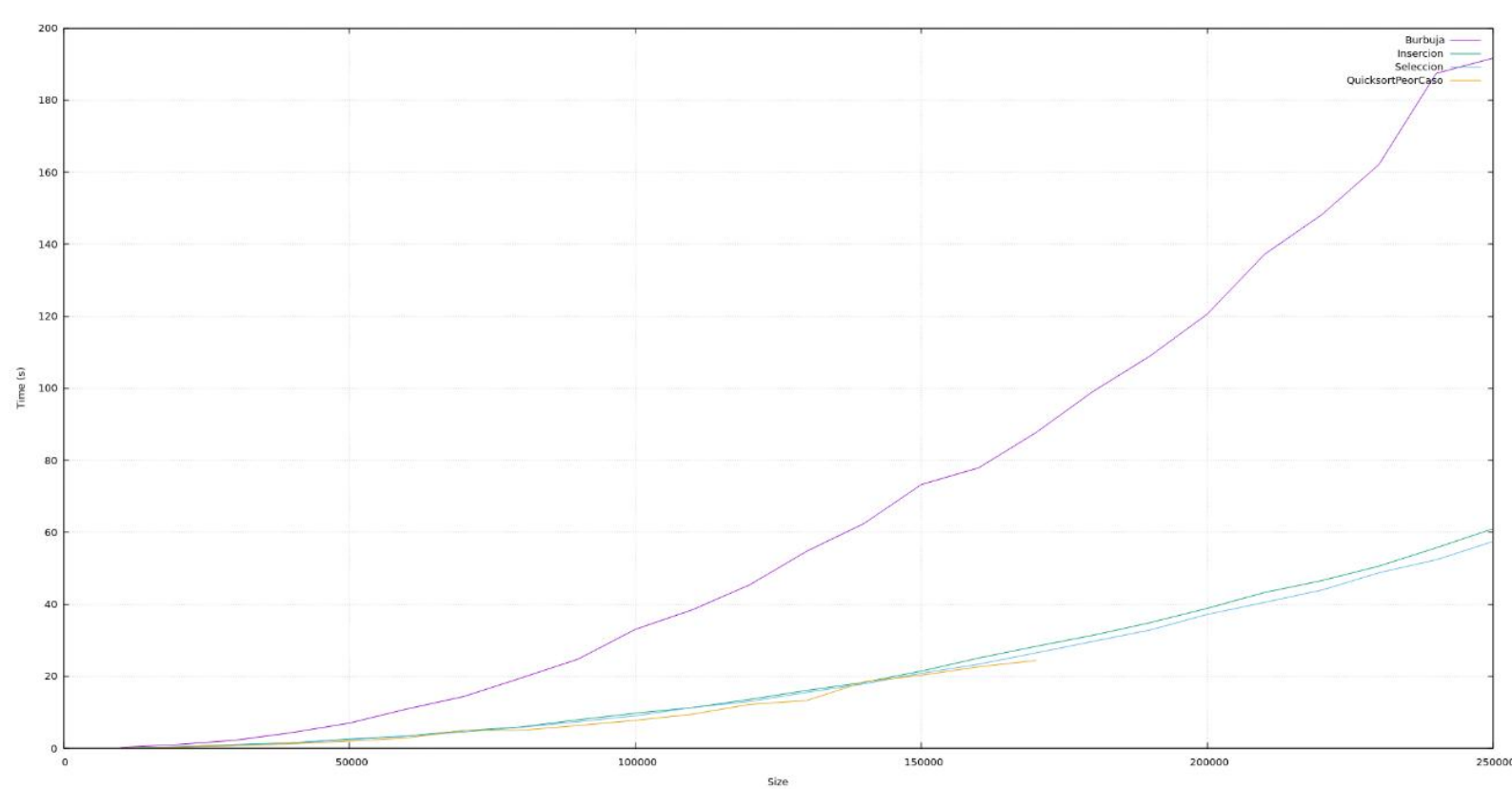
Burbuja



Inserción



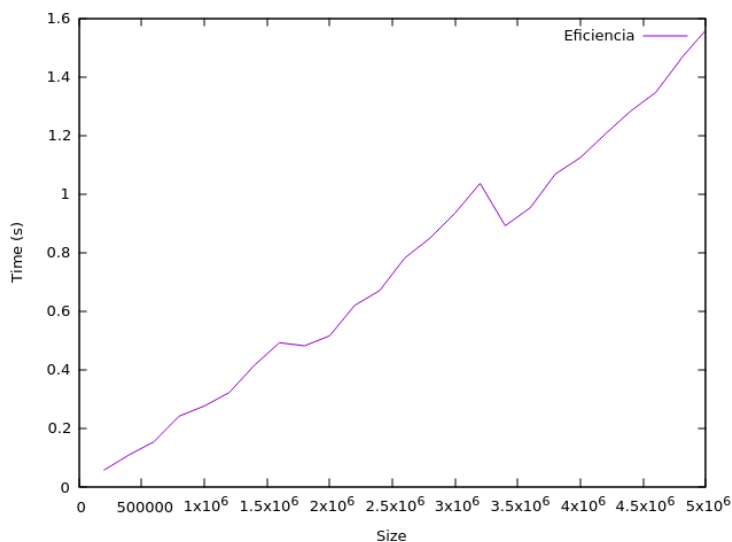
Selección



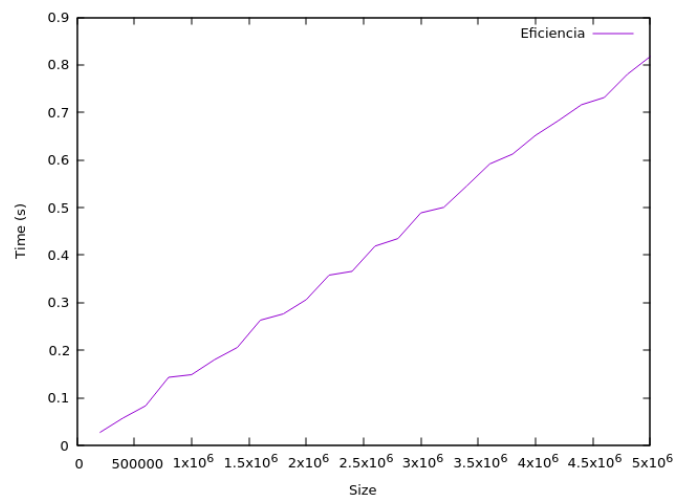
Grupo 1: $O(n^2)$

Como se puede observar en la ilustración del Grupo 1, se ha incluido el peor caso del Quicksort, el cual se degrada a una eficiencia cuadrática. Este caso ocurre al suministrarle un vector ya ordenado, debido a que hemos podido observar en la implementación del código que se elige como pivote siempre el primer elemento, en este caso será siempre el elemento mínimo, por lo que las particiones quedan desequilibradas, aumentando significativamente su tiempo de ejecución.

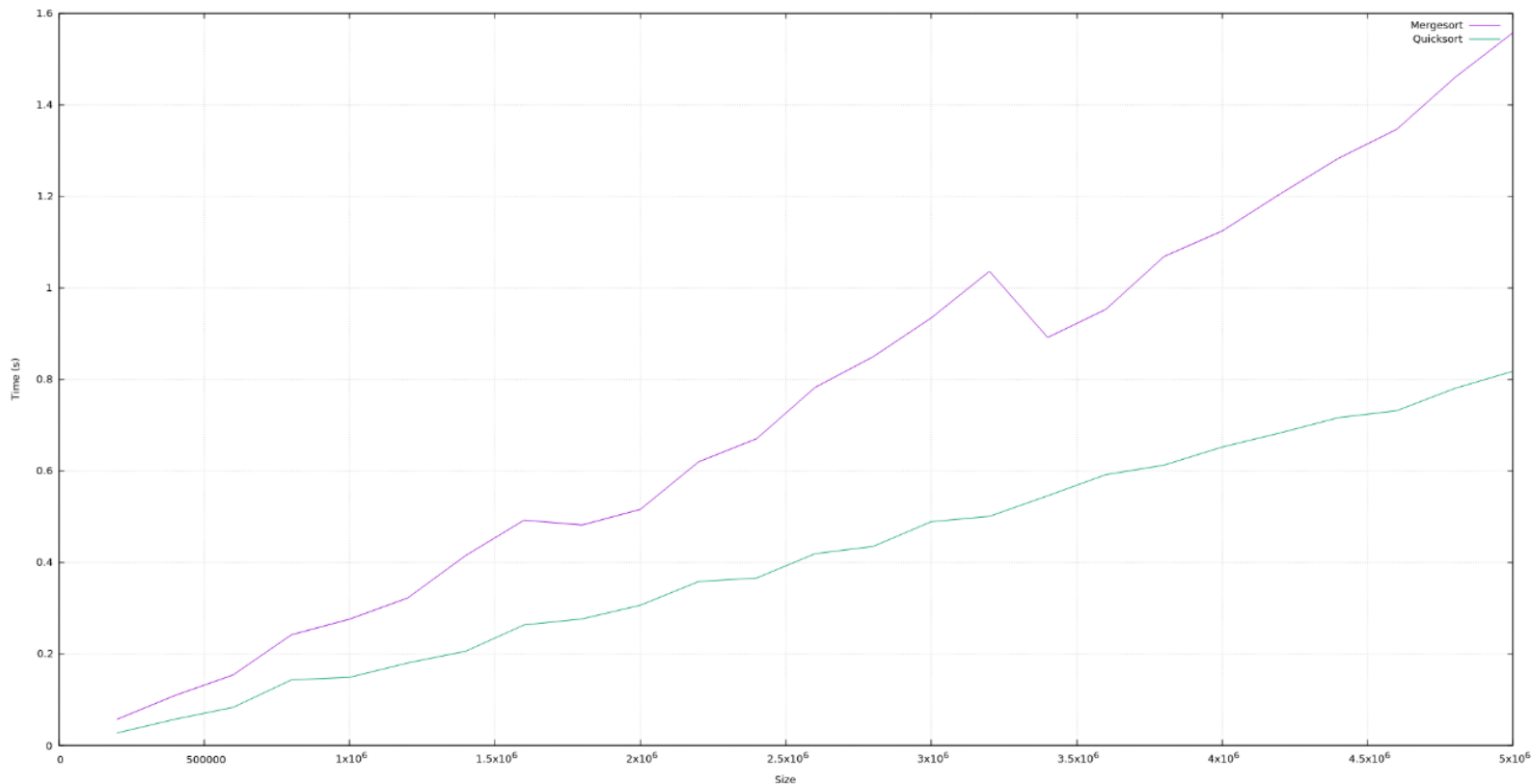
3.2. Lineal – Logarítmico



Mergesort



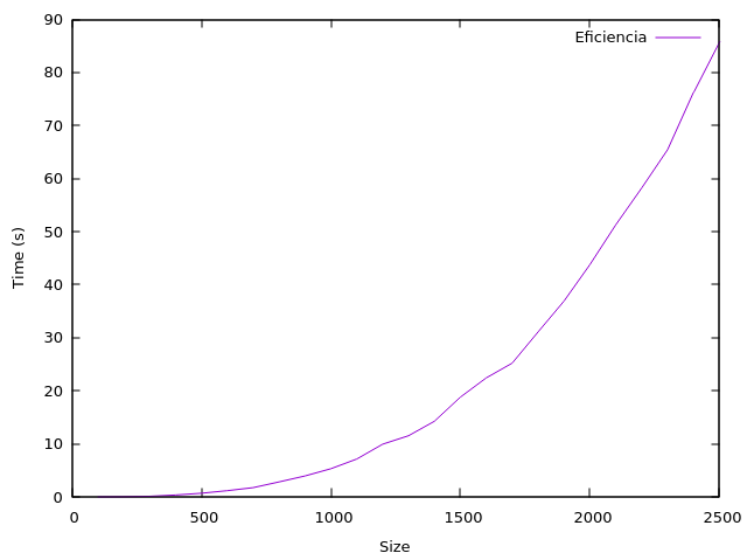
Quicksort



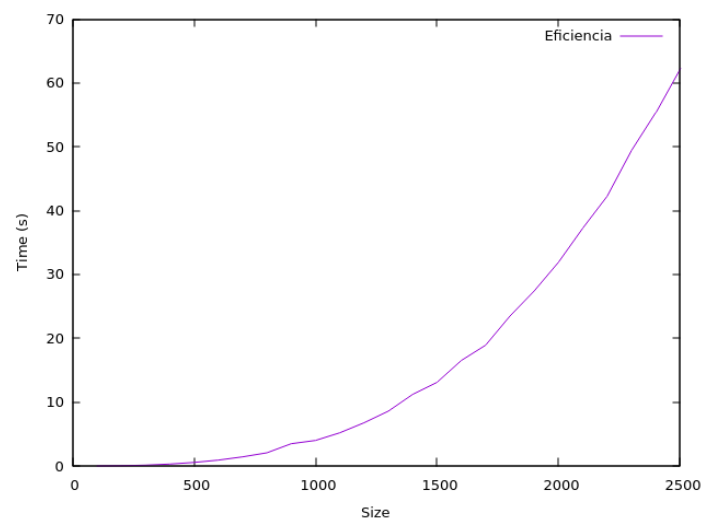
Grupo 2: $O(n \log(n))$

En esta sección queremos destacar el pico que se genera en el gráfico del mergesort cuando N supera los tres millones. Aunque, los gráficos no son curvas perfectas, esta anomalía es muy notable. Su causa radica en como el mergesort fusiona las mitades, ya que necesita reservar memoria dinámica para crear vectores auxiliares. Esta reserva masiva de memoria desborda la memoria caché y puede influir en la paginación del sistema operativo provocando ese salto en el tiempo.

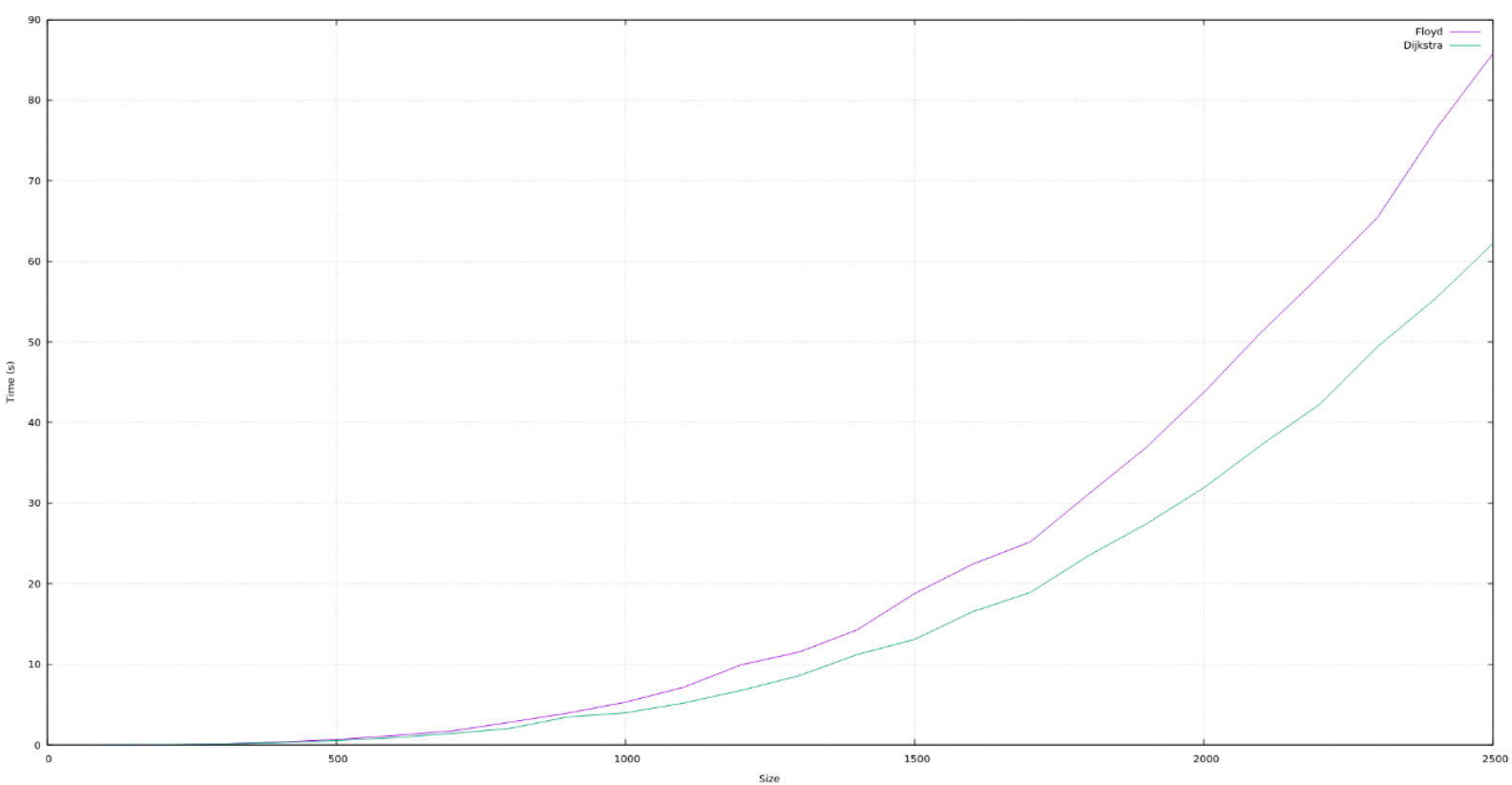
3.3.Cúbico



Floyd

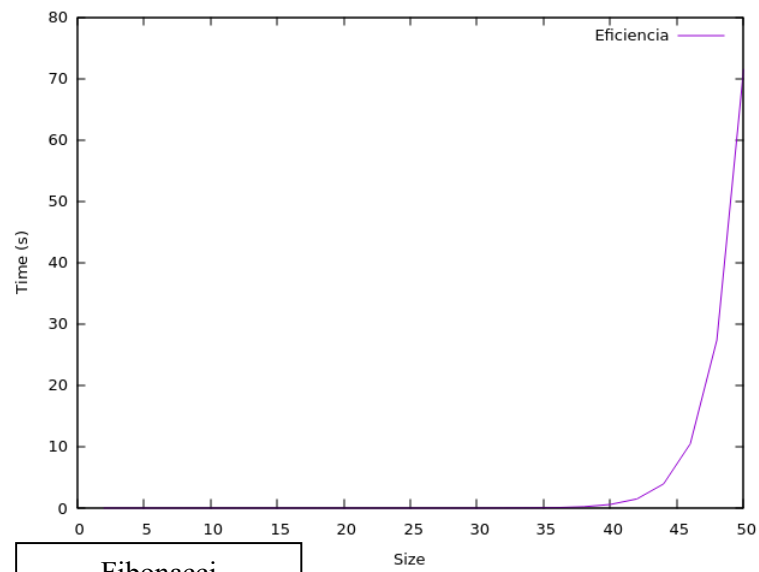


Dijkstra

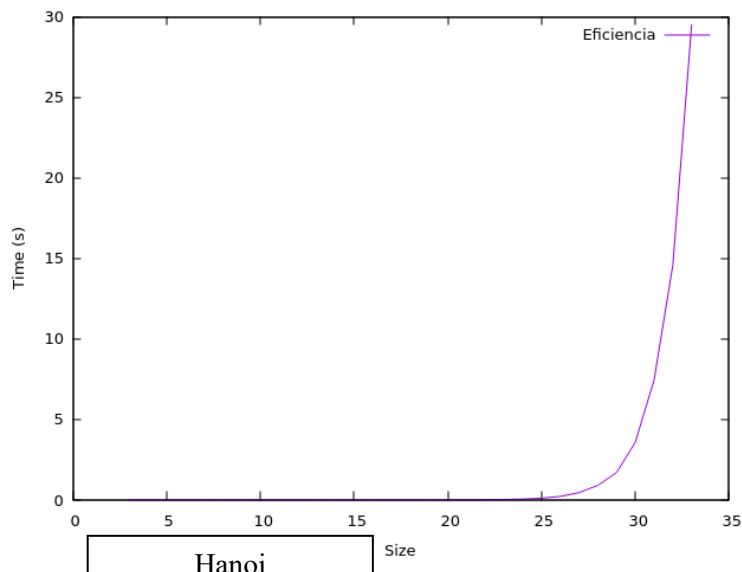


Grupo 3: $O(n^3)$

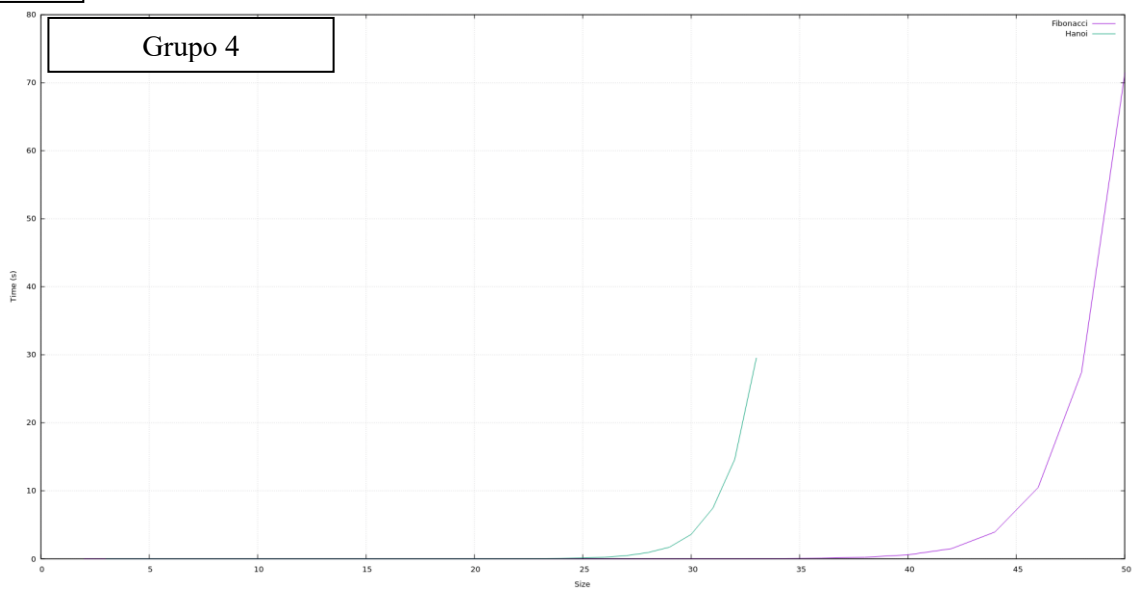
3.4.Exponencial



Fibonacci



Hanoi

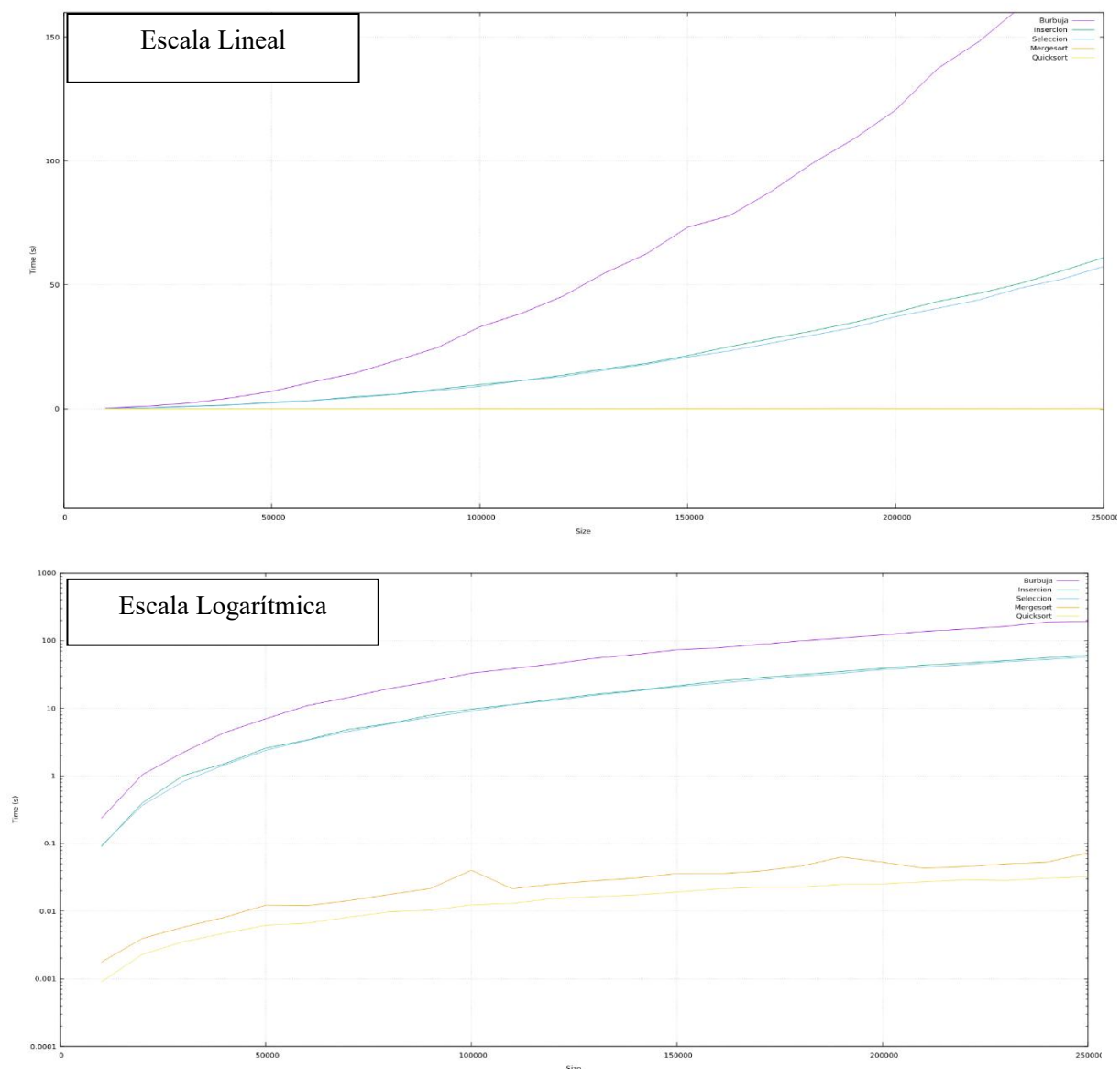


Grupo 4

3.5. Algoritmos de ordenación

Tal y como se sugiere en el guion de prácticas, se ha generado una gráfica conjunta para todos los algoritmos que realizan la misma tarea (ordenación de vectores). Siguiendo las indicaciones, se han adaptado las ejecuciones de los algoritmos de orden cuasi lineal (Mergesort y Quicksort) para que su tamaño inicial, final y salto coincidan exactamente con los utilizados en los algoritmos de orden cuadrático.

Para poder apreciar visualmente las abismales diferencias de rendimiento entre las distintas complejidades asintóticas, se incluyen a continuación dos representaciones. La primera muestra los datos en una escala lineal estándar. La segunda representación aplica una escala logarítmica en el eje de ordenadas (eje Y, mediante Gnuplot), lo que permite separar visualmente todas las series y evidenciar la superioridad del orden $O(n \log(n))$ frente al $O(n^2)$.

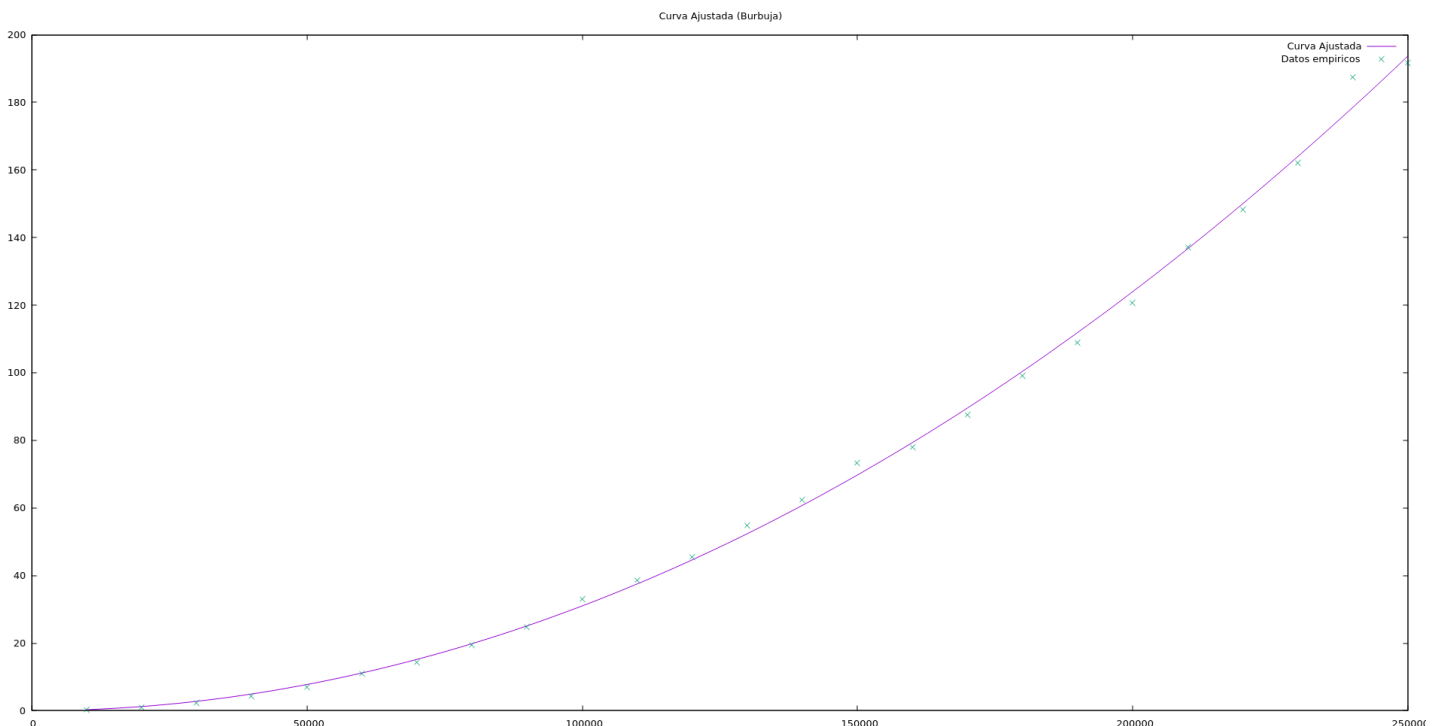


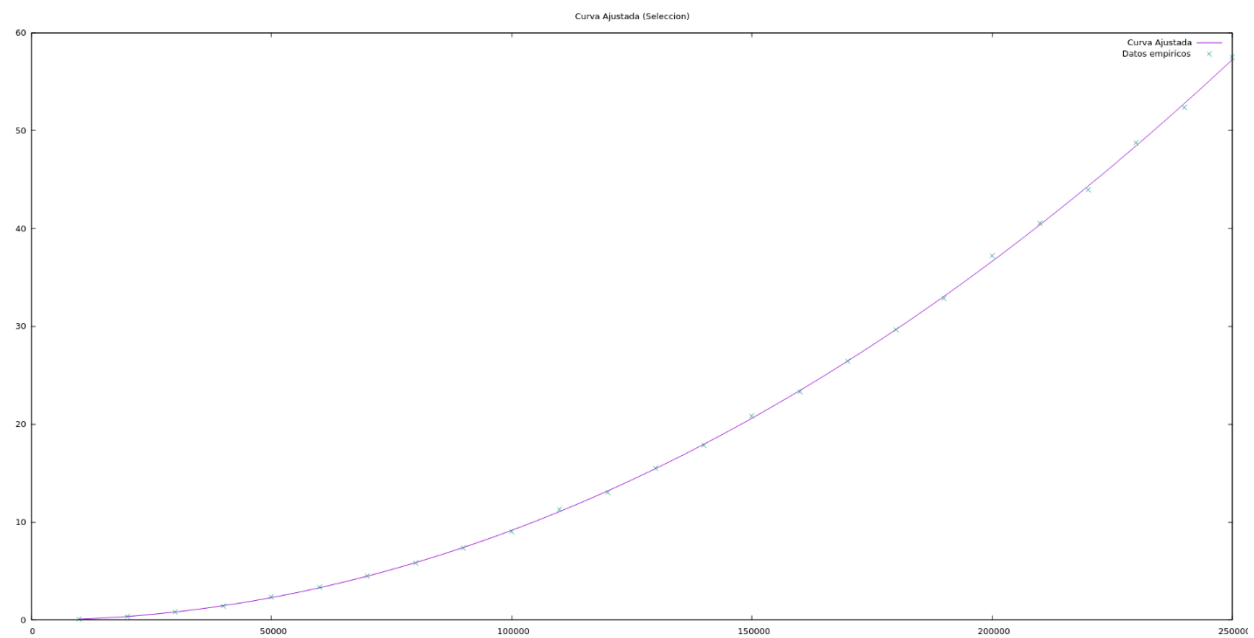
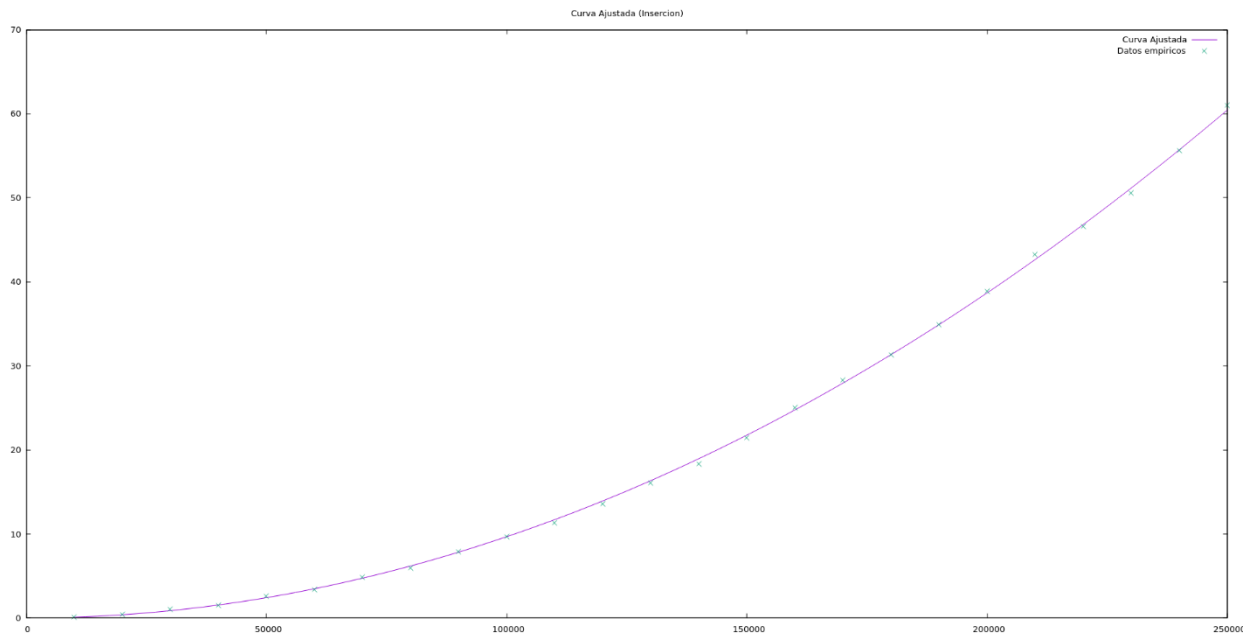
4. Eficiencia Híbrida

Una vez obtenidos los datos empíricos de cada algoritmo, hemos usado gnuplot para ajustar por mínimos cuadrados la eficiencia híbrida. Como se indica en el guion de prácticas solo hemos aplicado ajustes en el término dominante. A continuación, mostraremos una tabla resumen (datos del *ordenador1*) con la función que hemos utilizado para cada algoritmo, la constante oculta que calcula gnuplot (a_0) y el margen de error asociado al ajuste. Para obtener la variable oculta para cada caso hemos usado los comandos de gnuplot que se indican en el guion. Primero, definir la función (que varía para cada grupo). Posteriormente hemos tenido que ajustar la función con los datos, usando el comando *fit f(x) "data" via a0*. Por último, hemos dibujado (comando *plot*) tanto la función (como puntos) con los datos empíricos (como línea).

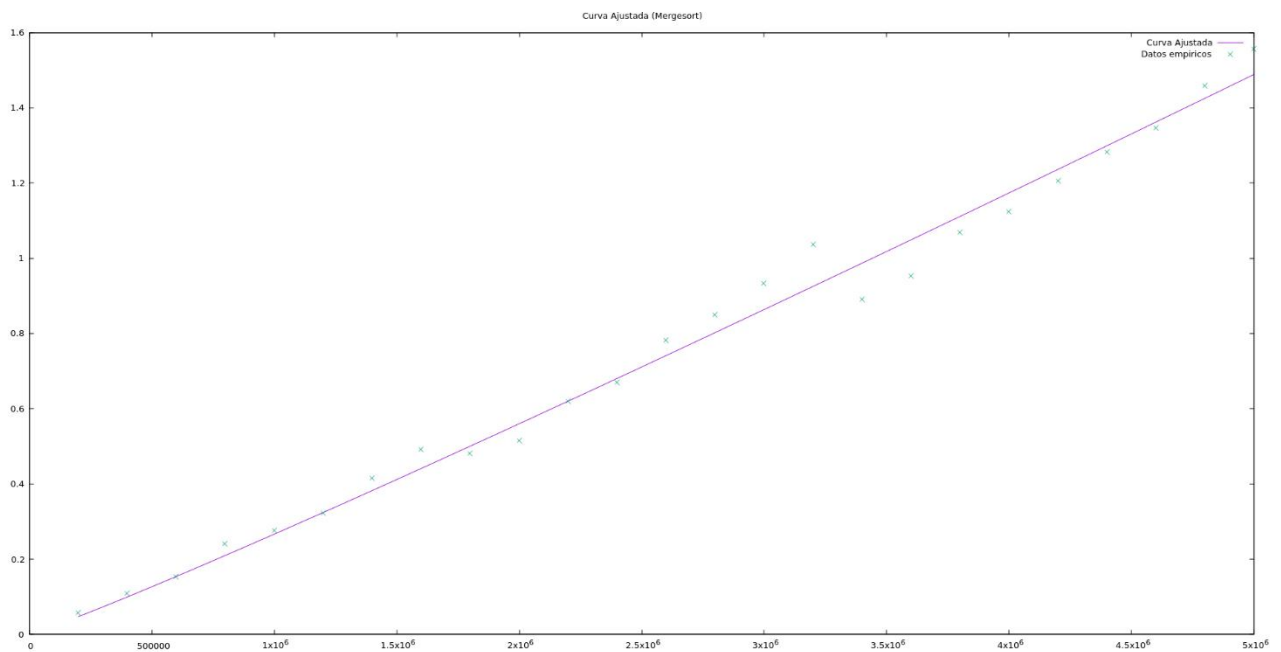
Algoritmo	Eficiencia	F(x)	A ₀	Margen de Error
Burbuja	$O(n^2)$	$f(x) = a_0 \cdot x^2$	3.10064e-09	0.5434%
Inserción			9.67492e-10	0.2194%
Selección			9.16356e-10	0.1546%
Mergesort	$O(n \log(n))$	$f(x) = a_0 \cdot x \cdot \log x$	1.93037e-08	1.138%
Quicksort			1.06451e-08	0.4342%
Floyd	$O(n^3)$	$f(x) = a_0 \cdot x^3$	5.45761e-09	0.3136%
Dijkstra			4.00065e-09	0.2274%
Fibonacci	$O(1.618^n)$	$f(x) = a_0 \cdot 1.618^x$	2.54198e-09	0.03798%
Hanoi	$O(2^n)$	$f(x) = a_0 \cdot 2^x$	3.42489e-09	0.1332%

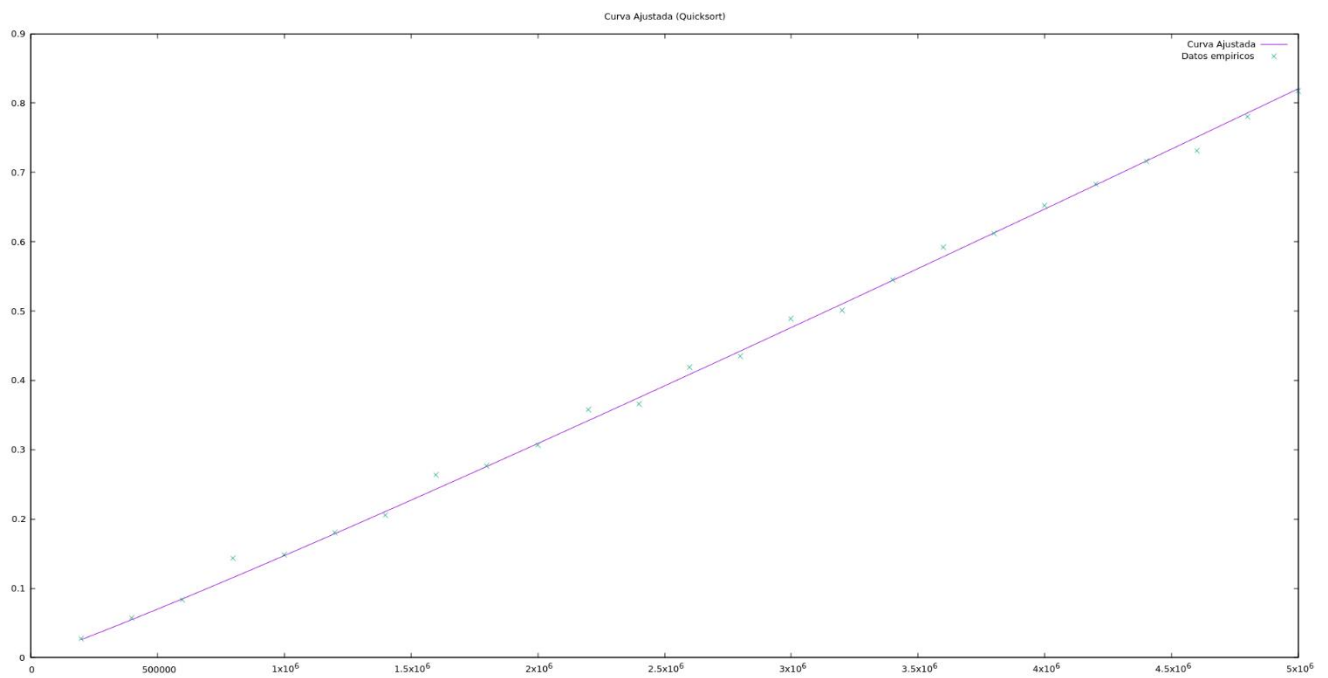
4.1. Cuadrático



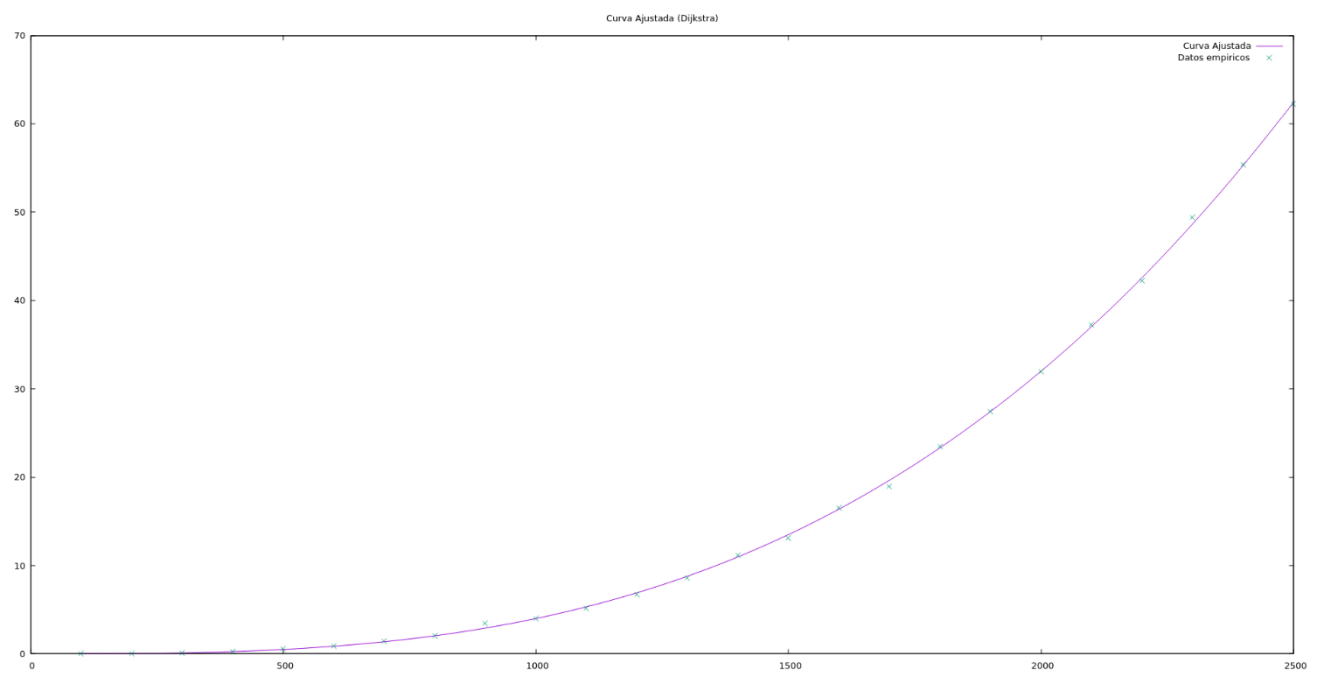
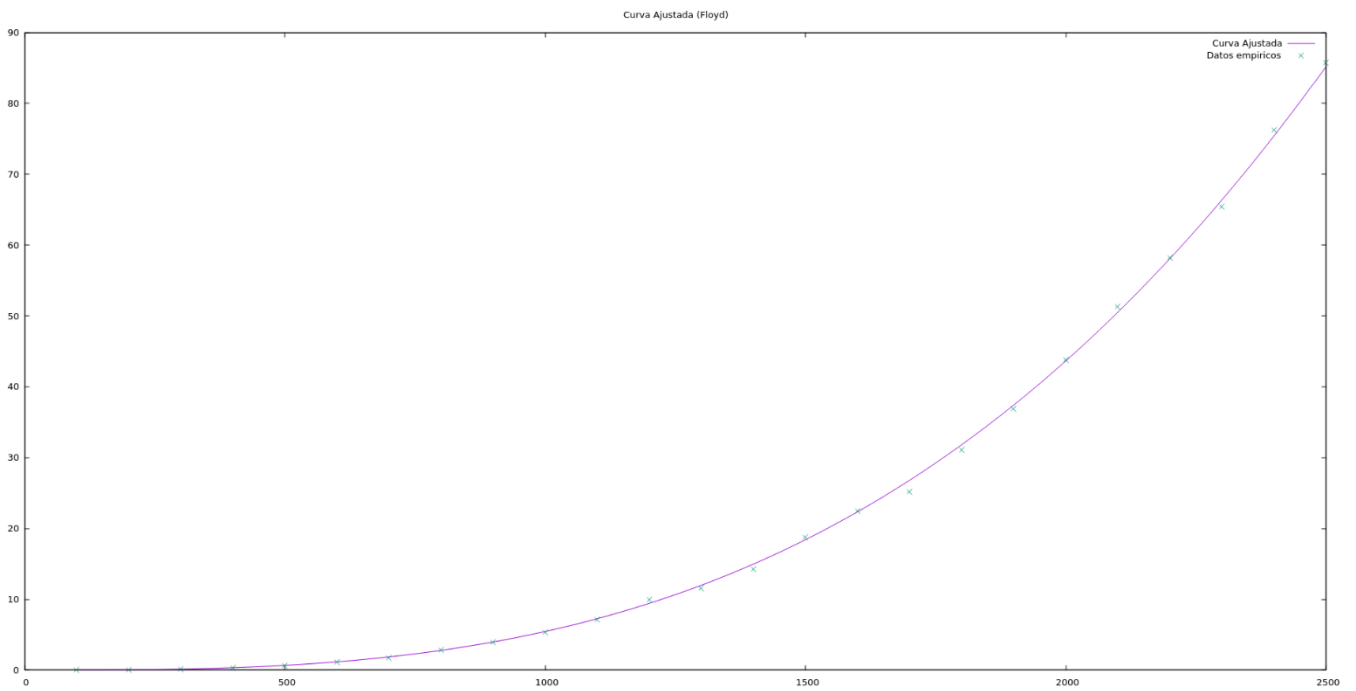


4.2.Lineal – Logarítmico



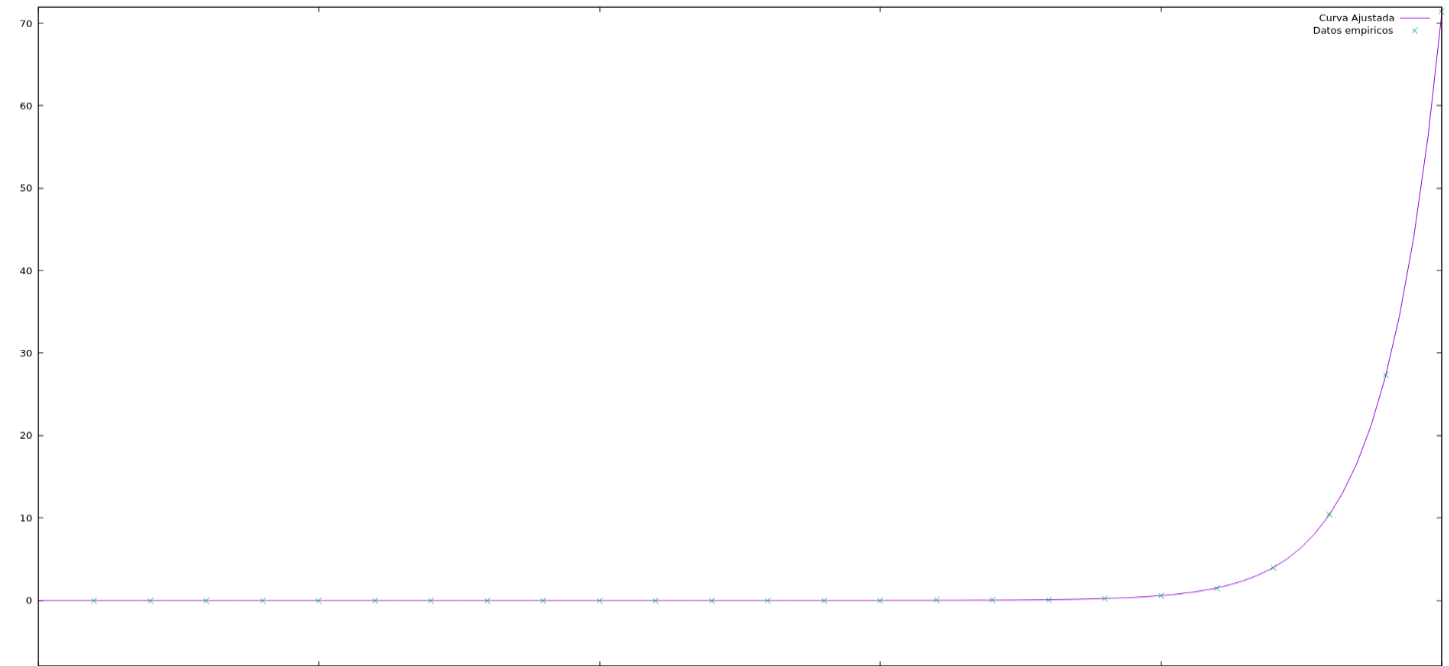


4.3.Cúbico

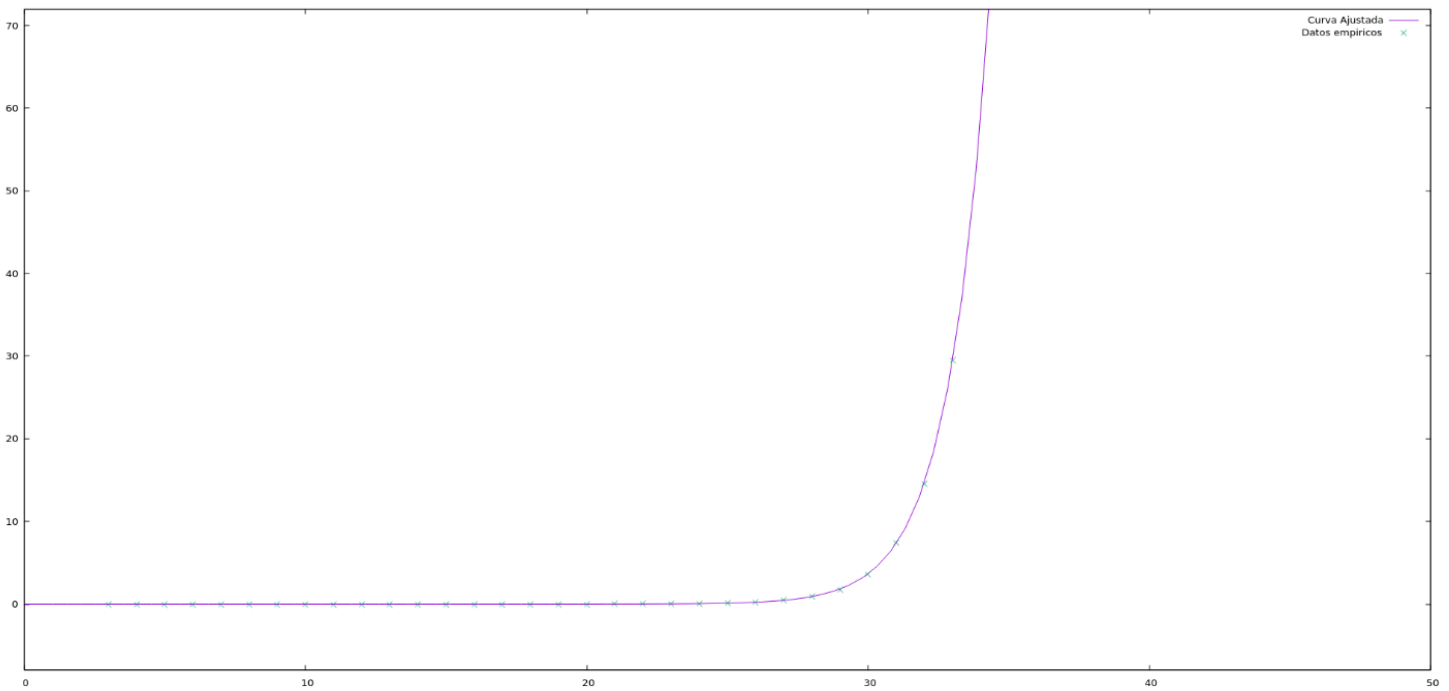


4.4.Exponencial

Curva Ajustada (Fibonacci)

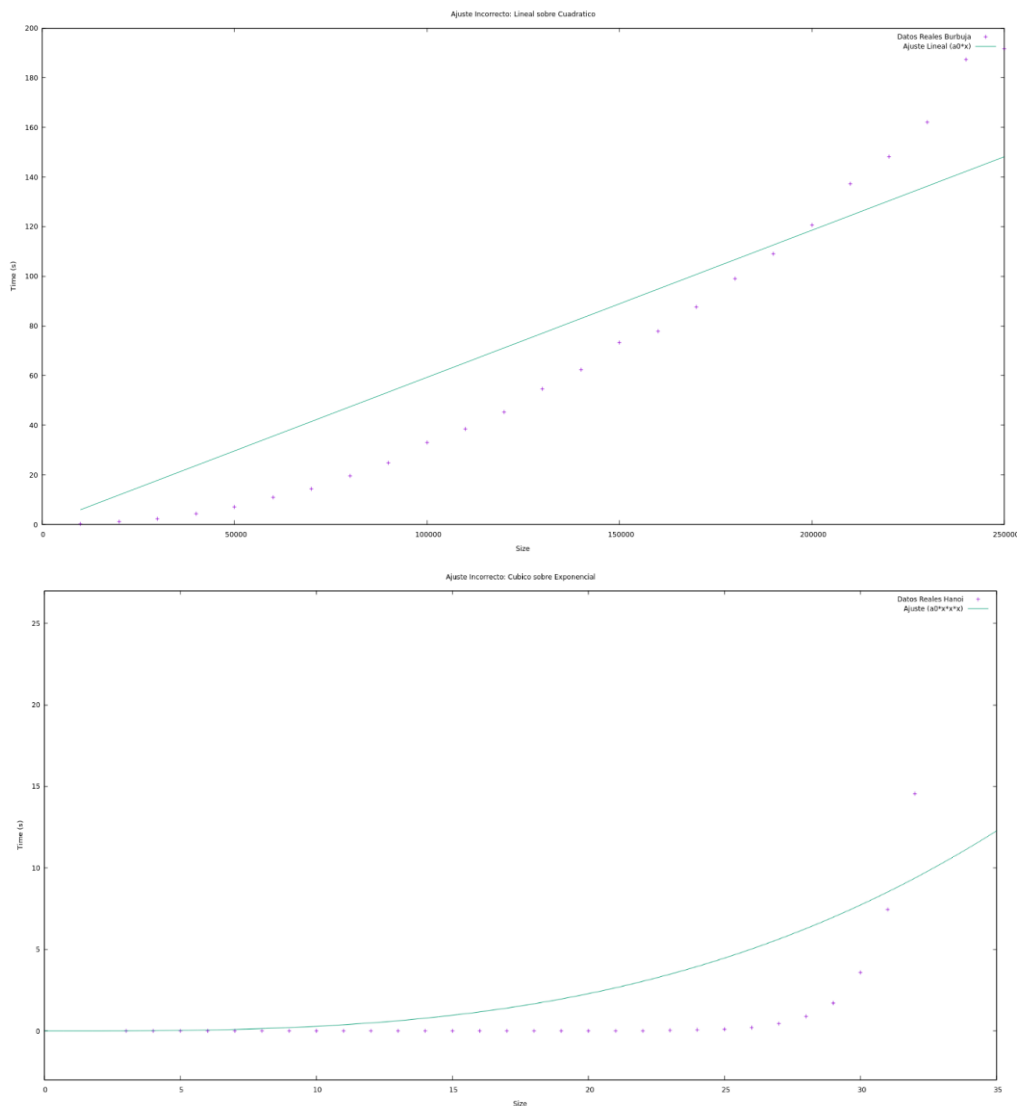


Curva Ajustada (Hanoi)



4.5. Ajuste Incorrecto

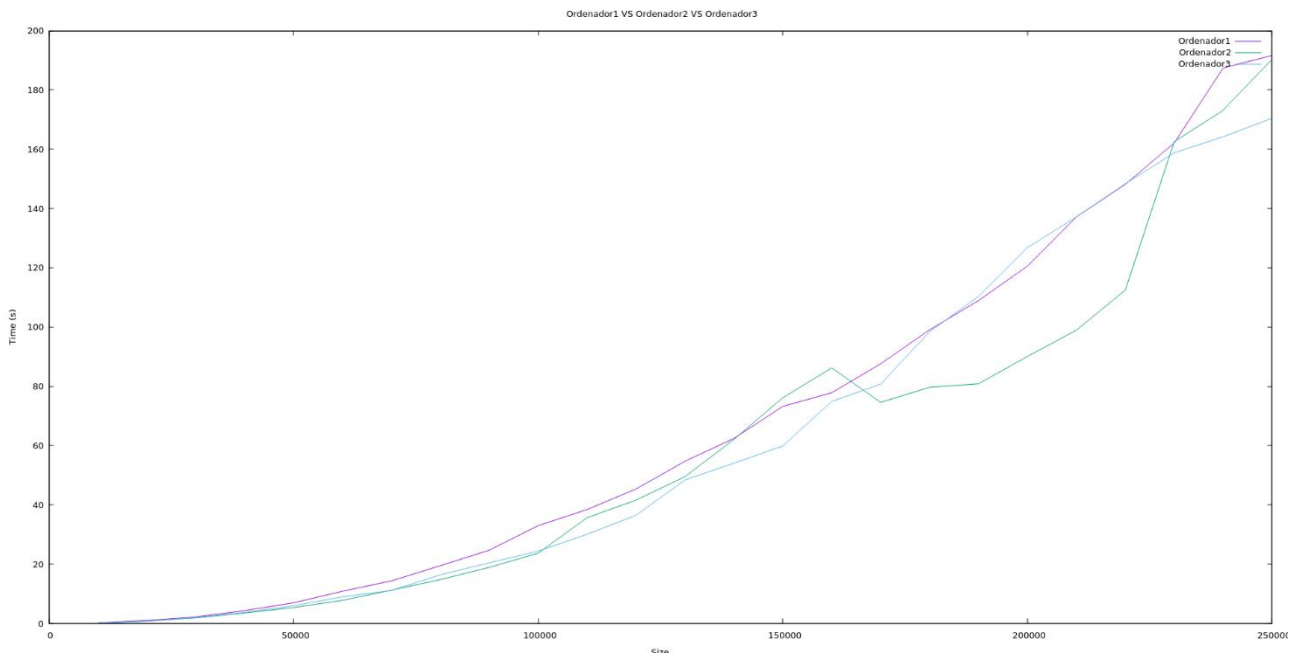
En este apartado hemos querido demostrar que la eficiencia teórica de los distintos algoritmos que hemos utilizado es correcta. Para ello, obligamos a Gnuplot que intente ajustar los datos empíricos de los algoritmos con una función que no es la teórica. Por ejemplo: para el método burbuja ($O(n^2)$) hemos utilizado una función lineal; mientras que para Hanoi ($O(2^n)$) hemos optado por una función cúbica. Tras ejecutar los comandos pertinentes, obtenemos las gráficas que nos dejan ver claramente que los datos empíricos no coinciden con la función que hemos definido, dejándonos ver que realmente la eficiencia teórica es cuadrática y exponencial respectivamente.



5. Principio de Invarianza

La finalidad de este apartado es demostrar el principio de invarianza, el cual establece que diferentes implementaciones del mismo algoritmo (variando hardware, optimización, lenguaje...) no alteran el orden de eficiencia asintótica, sino que modifican únicamente a constantes multiplicativas ocultas, en nuestro caso $a\theta$.

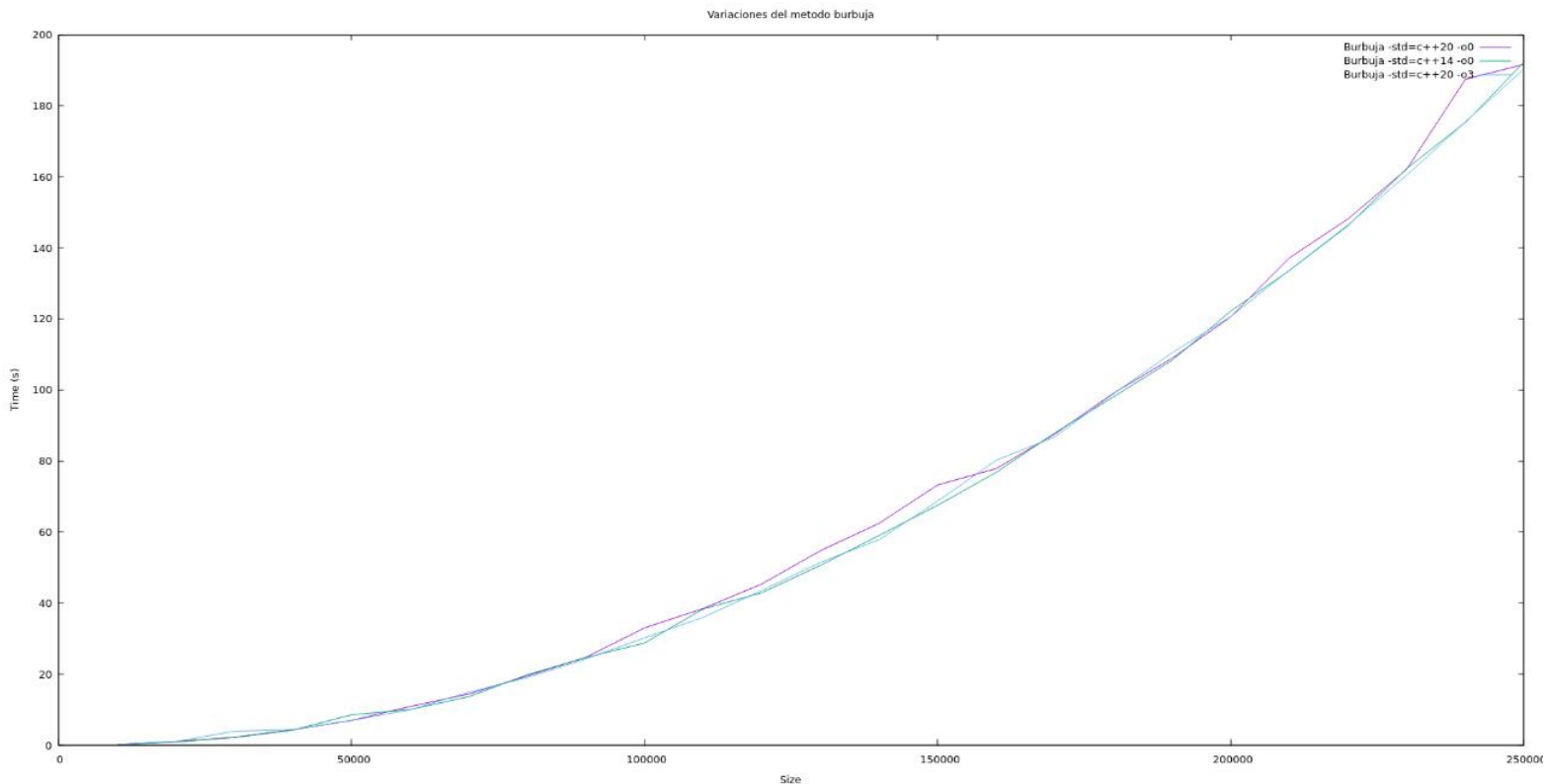
En la imagen siguiente hemos comparado el algoritmo burbuja en los tres ordenadores descritos al principio. Aunque se pueden apreciar diferencias entre los tres equipos, la forma geométrica de las curvas sigue siendo una parábola. Esto demuestra que el hardware afecta al rendimiento absoluto pero la eficiencia asintótica permanece invariable ($O(n^2)$).



Además de realizar las pruebas en diferentes equipos, hemos querido comprobar qué pasaría si cambiamos tanto el estándar de compilación como la optimización del compilador. Como hemos indicado anteriormente, hemos compilado todos los algoritmos con `C++20` y sin optimización; pero para ver si hay diferencias hemos vuelto a compilar el método burbuja con `-O3` y `C++14`.

Como se observa en la imagen siguiente, a pesar de que la optimización `-O3` hace que el compilador sea muy inteligente no puede “hacer magia” en algoritmos como el método burbuja, debido a su gran dependencia de datos y saltos impredecibles. Y también, tenemos que mencionar que el estándar en estos algoritmos, al tratarse

de instrucciones básicas y sin tener librerías modernas como *vector*; casi no influye en la eficiencia.



6. Conclusiones

Tras realizar todos los experimentos, mediciones y análisis gráficos de esta práctica, podemos extraer las siguientes conclusiones principales:

- **La validez de la teoría en el mundo real:** Hemos comprobado empíricamente que la teoría de la complejidad asintótica se cumple a la perfección. Al superponer los datos medidos por nuestro reloj con las funciones matemáticas generadas por Gnuplot, hemos visto cómo las curvas encajan. Dos algoritmos del mismo orden (como Burbuja y Selección) tienen la misma forma de crecimiento, diferenciándose únicamente en su constante oculta.
- **El Principio de Invarianza:** Hemos demostrado que mejorar el entorno (usando un ordenador con más memoria RAM o activando optimizaciones agresivas del compilador como -O3) reduce el tiempo total de ejecución, pero no altera la forma de la curva. Un algoritmo de orden $O(n^2)$ siempre crecerá de forma cuadrática.

- **La importancia del ajuste matemático:** Al realizar ajustes deliberadamente incorrectos (como intentar encajar la Burbuja en una línea recta), hemos visto cómo la gráfica se desvía y los errores residuales se disparan, lo que confirma matemáticamente que nuestras eficiencias teóricas eran las correctas.

7. Anexo

BURBUJA		INSERCIÓN		SELECCIÓN	
K	1	K	1	K	1
eficiencia	$O(n^2)$	eficiencia	$O(n^2)$	eficiencia	$O(n^2)$
n inicio	10000	n inicio	10000	n inicio	10000
n final	250000	n final	250000	n final	250000
salto	10000	salto	10000	salto	10000
Size	Time	Size	Time	Size	Time
10000	0,234794	10000	0,0895088	10000	0,0926665
20000	1,04052	20000	0,398316	20000	0,366636
30000	2,23383	30000	1,01391	30000	0,82915
40000	4,36943	40000	1,51565	40000	1,45444
50000	6,99269	50000	2,56619	50000	2,38226
60000	10,89380	60000	3,38705	60000	3,35747
70000	14,38110	70000	4,84942	70000	4,51756
80000	19,51710	80000	5,92285	80000	5,83566
90000	24,75660	90000	7,91603	90000	7,38267
100000	33,02450	100000	9,72275	100000	9,02811
110000	38,48530	110000	11,3138	110000	11,3228
120000	45,40300	120000	13,5769	120000	13,0154
130000	54,74290	130000	16,0562	130000	15,5417
140000	62,44100	140000	18,3071	140000	17,8745
150000	73,25550	150000	21,4403	150000	20,8405
160000	77,87150	160000	25,0498	160000	23,3534
170000	87,62160	170000	28,3047	170000	26,4659
180000	99,09330	180000	31,3775	180000	29,6803
190000	108,95500	190000	34,8838	190000	32,859
200000	120,59800	200000	38,8736	200000	37,2027
210000	137,19100	210000	43,26	210000	40,5139
220000	148,19200	220000	46,5677	220000	43,9388
230000	162,13400	230000	50,5861	230000	48,7548
240000	187,42300	240000	55,6701	240000	52,3398
250000	191,16400	250000	60,9891	250000	57,4721

MERGESORT		QUICKSORT		FLOYD		DIJKSTRA	
K	40	K	40	K	1	K	1
eficiencia	$O(n \log(n))$	eficiencia	$O(n \log(n))$	eficiencia	$O(n^3)$	eficiencia	$O(n^3)$
n inicio	200000	n inicio	200000	n inicio	100	n inicio	100
n final	5000000	n final	5000000	n final	2500	n final	2500
salto	200000	salto	200000	salto	100	salto	100
Size	Time	Size	Time	Size	Time	Size	Time
200000	0,0570308	200000	0,0271142	100	0,004999	100	0,00662222
400000	0,109314	400000	0,057372	200	0,0324787	200	0,0390569
600000	0,154183	600000	0,0833691	300	0,11414	300	0,124659
800000	0,241808	800000	0,143233	400	0,349698	400	0,28293
1000000	0,276015	1000000	0,14882	500	0,672359	500	0,544439
1200000	0,322263	1200000	0,180259	600	1,16117	600	0,906522
1400000	0,415192	1400000	0,205947	700	1,73185	700	1,42347
1600000	0,492166	1600000	0,263279	800	2,80944	800	2,0524
1800000	0,481582	1800000	0,276594	900	3,94125	900	3,47148
2000000	0,515854	2000000	0,3065	1000	5,30406	1000	3,98905
2200000	0,619687	2200000	0,358032	1100	7,14069	1100	5,18598
2400000	0,670419	2400000	0,366	1200	9,9398	1200	6,76147
2600000	0,781972	2600000	0,41895	1300	11,5195	1300	8,58594
2800000	0,849145	2800000	0,434904	1400	14,2649	1400	11,1998
3000000	0,934009	3000000	0,488857	1500	18,7913	1500	13,0953
3200000	1,0359	3200000	0,50064	1600	22,4267	1600	16,5274
3400000	0,89134	3400000	0,545431	1700	25,2009	1700	18,9243
3600000	0,952847	3600000	0,591675	1800	31,0937	1800	23,4756
3800000	1,06813	3800000	0,612495	1900	36,9075	1900	27,4123
4000000	1,124	4000000	0,651919	2000	43,775	2000	31,9293
4200000	1,20519	4200000	0,68292	2100	51,2795	2100	37,2714
4400000	1,28277	4400000	0,716224	2200	58,2242	2200	42,2559
4600000	1,34609	4600000	0,731423	2300	65,4529	2300	49,4139
4800000	1,4592	4800000	0,780085	2400	76,2729	2400	55,3795
5000000	1,55719	5000000	0,817689	2500	85,7852	2500	62,241

FIBONACCI		HANOI		HANOI	
K	3	K	3	K	3
eficiencia	$1,618^n$	eficiencia	2^n	eficiencia	2^n
n inicio	2	n inicio	3	n inicio	3
n final	50	n final	33	n final	33
salto	2	salto	1	salto	1
Size	Time	Size	Time	Size	Time
2	3,80E-08	3	7,83E-08	28	0,909462
4	7,40E-08	4	1,11E-07	29	1,70867
6	1,10E-07	5	3,61E-07	30	3,58649
8	2,73E-07	6	5,94E-07	31	7,45097
10	4,43E-07	7	1,03E-06	32	14,5617
12	1,22E-06	8	1,96E-06	33	29,4894
14	1,38E-05	9	2,97E-06		
16	5,98E-06	10	5,65E-06		
18	1,77E-05	11	1,09E-05		
20	8,82E-05	12	2,79E-05		
22	0,000120256	13	5,12E-05		
24	0,000331264	14	7,55E-05		
26	0,000772847	15	0,000155711		
28	0,00199861	16	0,000254072		
30	0,00469672	17	0,000498846		
32	0,0122464	18	0,0010127		
34	0,0312265	19	0,00187653		
36	0,084536	20	0,00384032		
38	0,221887	21	0,00711141		
40	0,579743	22	0,0150553		
42	1,49048	23	0,0369421		
44	3,93199	24	0,0585448		
46	10,4738	25	0,115819		
48	27,4119	26	0,228436		
50	71,4193	27	0,459922		